

Objetivo do Desafio

O seu objetivo é construir um componente React que simule uma busca de dados, utilizando dois hooks muito importantes no React:

-  **useState** → Para armazenar e controlar estados no seu componente.
 -  **useEffect** → Para executar ações que acontecem fora da renderização, como simular uma busca de dados.
-

O que você vai construir?

Imagine que você precisa exibir informações de um usuário (ou qualquer outro dado) na sua aplicação. Mas esses dados demoram alguns segundos para carregar, como acontece na vida real quando acessamos uma API.

O desafio é simular esse processo de carregamento de dados e exibir uma mensagem enquanto os dados estão sendo buscados. Depois disso, você deverá exibir as informações na tela de forma organizada.

O que precisa ter no seu código?

♦ 1. Estado de carregamento

- Crie um estado que começa como `true` e serve para controlar se os dados estão sendo carregados.
 - Exemplo de cenário: mostrar a mensagem “ Carregando dados...” enquanto o carregamento estiver ativo.
-

♦ 2. Estado dos dados

- Crie um segundo estado para armazenar as informações dos dados simulados.
 - Você pode simular qualquer dado, como informações de um usuário, produto, filme, livro, etc.
-

♦ 3. **useEffect** para simular a busca

- Utilize o **useEffect** com um array de dependências vazio (`[]`), isso fará com que o código dentro dele execute **apenas uma vez**, no momento em que o componente for montado.
 - Dentro do **useEffect**, simule o carregamento usando a função `setTimeout` para esperar alguns segundos antes de atualizar o estado dos dados.
 - Após o tempo de espera, atualize o estado dos dados e mude o estado de carregamento para `false`.
-

♦ 4. **Renderização condicional**

- Enquanto os dados estão carregando (`isLoading === true`), exiba uma mensagem informando que os dados estão sendo carregados.
 - Quando o carregamento terminar, exiba os dados de forma organizada e clara na interface.
-

♦ 5. **Função de limpeza (Boas Práticas)**

- Adicione uma função de limpeza dentro do **useEffect** para garantir que, se o componente for desmontado antes do carregamento terminar, o timer seja cancelado corretamente.
 - Isso é uma boa prática para evitar erros ou vazamentos de memória na sua aplicação.
-

✨ **Dicas de ouro**

- ✓ Sempre declare seus hooks (`useState` e `useEffect`) dentro do seu componente, nunca fora dele.
- ✓ Use uma estilização simples (inline ou CSS externo) para deixar os dados mais organizados visualmente.
- ✓ Lembre-se que o **useEffect** roda efeitos colaterais, ou seja, ações que não fazem parte diretamente da renderização (ex.: buscas, manipulações externas, timers, etc.).

- ✓ Pratique a **renderização condicional**, mostrando uma coisa enquanto carrega e outra quando finaliza.
-

Habilidades que você estará desenvolvendo nesse exercício:

- Entender como funciona o ciclo de vida de um componente React.
- Utilizar os hooks `useEffect` e `useState` de forma prática.
- Trabalhar com simulações de chamadas assíncronas (como APIs).
- Implementar renderização condicional no React.
- Aplicar boas práticas com funções de limpeza dentro do `useEffect`.